

1. Smart Healthcare System – Bayesian Belief Network for Anomaly Detection 1.1

Objective

The objective is to develop a smart healthcare monitoring system that continuously receives vital-sign data from wearable sensors and detects abnormal patient conditions.

```
!pip install pgmpy
```

```
Collecting pgmpy
```

```
  Downloading pgmpy-1.1.2-py3-none-any.whl.metadata (13 kB)
```

```
Requirement already satisfied: huggingface_hub>=0.23 in /usr/local/lib/python3.12/dist-packages (from pgmpy)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-packages (from pgmpy)
Requirement already satisfied: numpy>=2.0 in /usr/local/lib/python3.12/dist-packages (from pgmpy)
Requirement already satisfied: scipy>=1.10 in /usr/local/lib/python3.12/dist-packages (from pgmpy)
Requirement already satisfied: scikit-learn>=1.2 in /usr/local/lib/python3.12/dist-packages (from pgmpy)
Requirement already satisfied: pandas>=1.5 in /usr/local/lib/python3.12/dist-packages (from pgmpy)
Requirement already satisfied: statsmodels>=0.14.5 in /usr/local/lib/python3.12/dist-packages (from pgmpy)
Requirement already satisfied: tqdm>=4.64 in /usr/local/lib/python3.12/dist-packages (from pgmpy)
Requirement already satisfied: pyparsing>=3.0 in /usr/local/lib/python3.12/dist-packages (from pgmpy)
Requirement already satisfied: joblib>=1.2 in /usr/local/lib/python3.12/dist-packages (from pgmpy)
Requirement already satisfied: opt_einsum>=3.3 in /usr/local/lib/python3.12/dist-packages (from pgmpy)
Collecting scikit-base>=0.12.4 (from pgmpy)
```

```
  Downloading scikit_base-1.1.0-py3-none-any.whl.metadata (8.8 kB)
```

```
Requirement already satisfied: click<9.0.0,>=8.4.2 in /usr/local/lib/python3.12/dist-packages (from scikit-base)
Requirement already satisfied: filelock>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from scikit-base)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from scikit-base)
Requirement already satisfied: hf-xet<2.0.0,>=1.5.1 in /usr/local/lib/python3.12/dist-packages (from scikit-base)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from scikit-base)
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.12/dist-packages (from scikit-base)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from scikit-base)
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-base)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from scikit-base)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from scikit-base)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from scikit-base)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-base)
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.12/dist-packages (from scikit-base)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from scikit-base)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from scikit-base)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from scikit-base)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from scikit-base)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from scikit-base)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from scikit-base)
Downloading pgmpy-1.1.2-py3-none-any.whl (2.4 MB)
```

```
----- 2.4/2.4 MB 27.9 MB/s eta 0:00:00
```

```
Downloading scikit_base-1.1.0-py3-none-any.whl (165 kB)
```

```
----- 165.4/165.4 kB 13.8 MB/s eta 0:00:00
```

```
Installing collected packages: scikit-base, pgmpy
```

```
Successfully installed pgmpy-1.1.2 scikit-base-1.1.0
```

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_ma

from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.estimators import MaximumLikelihoodEstimator
from pgmpy.inference import VariableElimination

/usr/local/lib/python3.12/dist-packages/pgmpy/estimators/__init__.py:4: FutureWa
    from .StructureScore import (

```

```

np.random.seed(42)

n = 1000

data = pd.DataFrame({
    'HeartRate': np.random.normal(80, 15, n),
    'BloodPressure': np.random.normal(120, 15, n),
    'SpO2': np.random.normal(97, 2, n),
    'Temperature': np.random.normal(36.8, 0.7, n),
    'RespiratoryRate': np.random.normal(18, 4, n)
})

# Create health condition
data['HealthCondition'] = (
    (data['HeartRate'] > 110) |
    (data['BloodPressure'] > 145) |
    (data['SpO2'] < 92) |
    (data['Temperature'] > 38) |
    (data['RespiratoryRate'] > 25)
).astype(int)

data.head()

```

	HeartRate	BloodPressure	SpO2	Temperature	RespiratoryRate	HealthConc
0	87.450712	140.990332	95.649643	35.464535	14.546026	
1	77.926035	133.869505	96.710963	36.197730	17.875186	
2	89.715328	120.894456	95.415160	36.510476	18.072067	
3	102.845448	110.295948	96.384077	38.121381	19.890521	
4	76.487699	130.473350	93.212771	37.189587	12.532567	

Next steps:

[Generate code with data](#)

[New interactive sheet](#)

```
data['HeartRate'] = pd.cut(
    data['HeartRate'],
    bins=[-np.inf, 60, 100, np.inf],
    labels=['Low', 'Normal', 'High']
)

data['BloodPressure'] = pd.cut(
    data['BloodPressure'],
    bins=[-np.inf, 90, 140, np.inf],
    labels=['Low', 'Normal', 'High']
)

data['SpO2'] = pd.cut(
    data['SpO2'],
    bins=[-np.inf, 92, 95, np.inf],
    labels=['Low', 'Moderate', 'Normal']
)

data['Temperature'] = pd.cut(
    data['Temperature'],
    bins=[-np.inf, 36, 38, np.inf],
    labels=['Low', 'Normal', 'High']
)

data['RespiratoryRate'] = pd.cut(
    data['RespiratoryRate'],
    bins=[-np.inf, 12, 25, np.inf],
    labels=['Low', 'Normal', 'High']
)

data['HealthCondition'] = data['HealthCondition'].map({
    0: 'Normal',
    1: 'Abnormal'
})

data.head()
```

	HeartRate	BloodPressure	SpO2	Temperature	RespiratoryRate	HealthCondition
0	Normal	High	Normal	Low	Normal	Normal
1	Normal	Normal	Normal	Normal	Normal	Normal
2	Normal	Normal	Normal	Normal	Normal	Normal
3	High	Normal	Normal	High	Normal	Abnormal
4	Normal	Normal	Moderate	Normal	Normal	Normal

Next steps:

[Generate code with data](#)

[New interactive sheet](#)

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.inference import VariableElimination
```

```
# Create Bayesian Network
```

```
model = DiscreteBayesianNetwork([
    ('HealthCondition', 'HeartRate'),
    ('HealthCondition', 'BloodPressure'),
    ('HealthCondition', 'SpO2'),
    ('HealthCondition', 'Temperature'),
    ('HealthCondition', 'RespiratoryRate')
])
```

```
# Maximum Likelihood estimation is the default
model.fit(data)
```

```
print("Bayesian Network successfully trained!")
```

```
# Display learned CPDs
```

```
print("\nLearned Probability Tables:\n")
```

```
for cpd in model.get_cpds():
    print(cpd)
    print()
```

```
Learned Probability Tables:
```

```
+-----+-----+
| HealthCondition(Abnormal) | 0.145 |
+-----+-----+
| HealthCondition(Normal)   | 0.855 |
+-----+-----+
```

```
+-----+-----+-----+
| HealthCondition | HealthCondition(Abnormal) | HealthCondition(Normal) |
+-----+-----+-----+
```

HeartRate(Normal)	0.7310344827586207	0.8315789473684211
-------------------	--------------------	--------------------

HealthCondition	HealthCondition(Abnormal)	HealthCondition(Normal)
BloodPressure(High)	0.4068965517241379	0.047953216374269005
BloodPressure(Low)	0.0	0.021052631578947368
BloodPressure(Normal)	0.593103448275862	0.9309941520467836

HealthCondition	HealthCondition(Abnormal)	HealthCondition(Normal)
SpO2(Low)	0.04827586206896552	0.0
SpO2(Moderate)	0.15862068965517243	0.14269005847953217
SpO2(Normal)	0.7931034482758621	0.8573099415204678

HealthCondition	HealthCondition(Abnormal)	HealthCondition(Normal)
Temperature(High)	0.3103448275862069	0.0
Temperature(Low)	0.14482758620689656	0.14502923976608187
Temperature(Normal)	0.5448275862068965	0.8549707602339182

HealthCondition	...	HealthCondition(Normal)
RespiratoryRate(High)	...	0.0
RespiratoryRate(Low)	...	0.08187134502923976
RespiratoryRate(Normal)	...	0.9181286549707602

```
inference = VariableElimination(model)
```

```
result = inference.query(
    variables=['HealthCondition'],
    evidence={
        'HeartRate': 'High',
        'BloodPressure': 'High',
        'SpO2': 'Low',
        'Temperature': 'High',
        'RespiratoryRate': 'High'
```

```

        }
    )

print(result)

```

```

+-----+-----+
| HealthCondition | phi(HealthCondition) |
+=====+=====+
| HealthCondition(Abnormal) | 1.0000 |
+-----+-----+
| HealthCondition(Normal) | 0.0000 |
+-----+-----+

```

```

result = inference.query(
    variables=['HealthCondition'],
    evidence={
        'HeartRate': 'Normal',
        'BloodPressure': 'Normal',
        'SpO2': 'Normal',
        'Temperature': 'Normal',
        'RespiratoryRate': 'Normal'
    }
)

print(result)

```

```

+-----+-----+
| HealthCondition | phi(HealthCondition) |
+=====+=====+
| HealthCondition(Abnormal) | 0.0458 |
+-----+-----+
| HealthCondition(Normal) | 0.9542 |
+-----+-----+

```

```

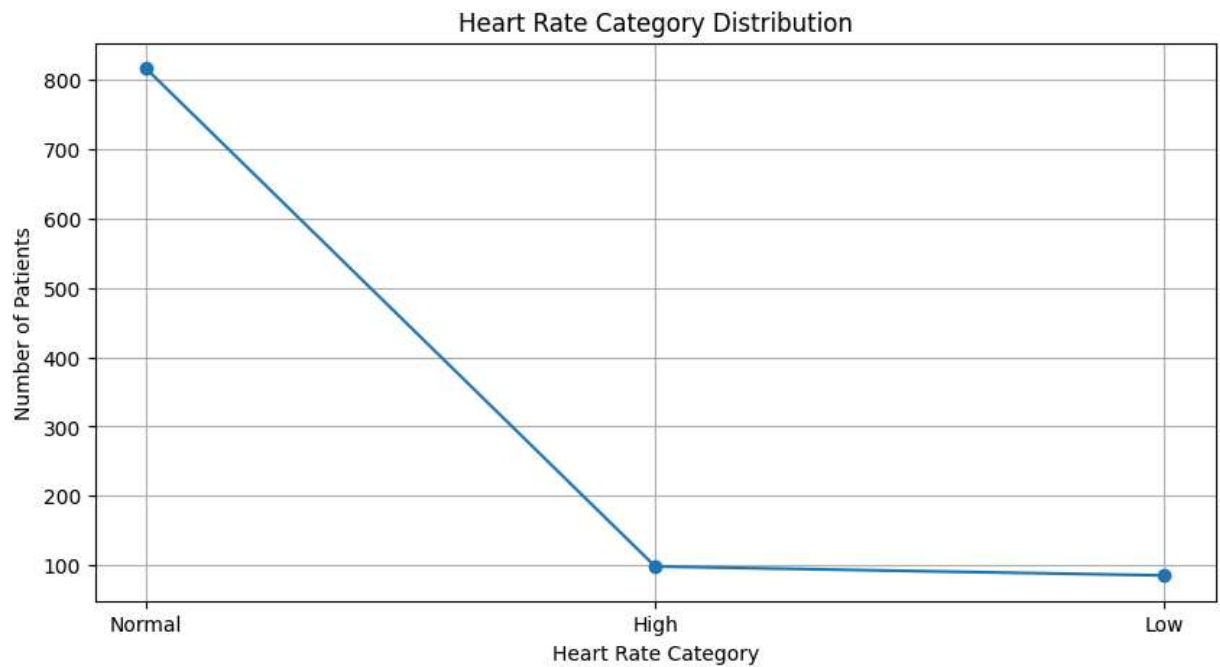
plt.figure(figsize=(10, 5))

plt.plot(data['HeartRate'].astype(str).value_counts(),
         marker='o')

plt.title("Heart Rate Category Distribution")
plt.xlabel("Heart Rate Category")
plt.ylabel("Number of Patients")
plt.grid(True)

plt.show()

```



2. Cybersecurity – Probabilistic Learning for Network Intrusion Detection 2.1

Objective

The objective is to develop an intrusion detection system (IDS) that distinguishes between:

Normal network traffic Malicious/intrusion traffic

using:

Naïve Bayes Expectation-Maximization (EM) Bayesian learning

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

from sklearn.mixture import GaussianMixture
```

```

np.random.seed(42)

n_normal = 1000
n_attack = 500

normal = pd.DataFrame({
    'PacketSize': np.random.normal(500, 80, n_normal),
    'Duration': np.random.normal(20, 5, n_normal),
    'Connections': np.random.normal(10, 3, n_normal),
    'FailedLogins': np.random.normal(1, 1, n_normal)
})

attack = pd.DataFrame({
    'PacketSize': np.random.normal(900, 150, n_attack),
    'Duration': np.random.normal(5, 2, n_attack),
    'Connections': np.random.normal(40, 10, n_attack),
    'FailedLogins': np.random.normal(15, 5, n_attack)
})

normal['Attack'] = 0
attack['Attack'] = 1

network_data = pd.concat([normal, attack], ignore_index=True)

network_data.head()

```

	PacketSize	Duration	Connections	FailedLogins	Attack	
0	539.737132	26.996777	7.974465	-0.907808	0	
1	488.938856	24.623168	9.566444	0.139615	0	
2	551.815083	20.298152	7.622740	0.586394	0	
3	621.842389	16.765316	9.076115	2.887688	0	
4	481.267730	23.491117	4.319156	1.556553	0	

Next steps:

[Generate code with network_data](#)

[New interactive sheet](#)

```

X = network_data.drop('Attack', axis=1)
y = network_data['Attack']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42,
    stratify=y
)

```



```
nb_model = GaussianNB()

nb_model.fit(X_train, y_train)

y_pred = nb_model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("Naïve Bayes Performance")
print("-----")
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
```

```
Naïve Bayes Performance
-----
Accuracy : 1.0
Precision: 1.0
Recall   : 1.0
F1 Score : 1.0
```

```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 5))

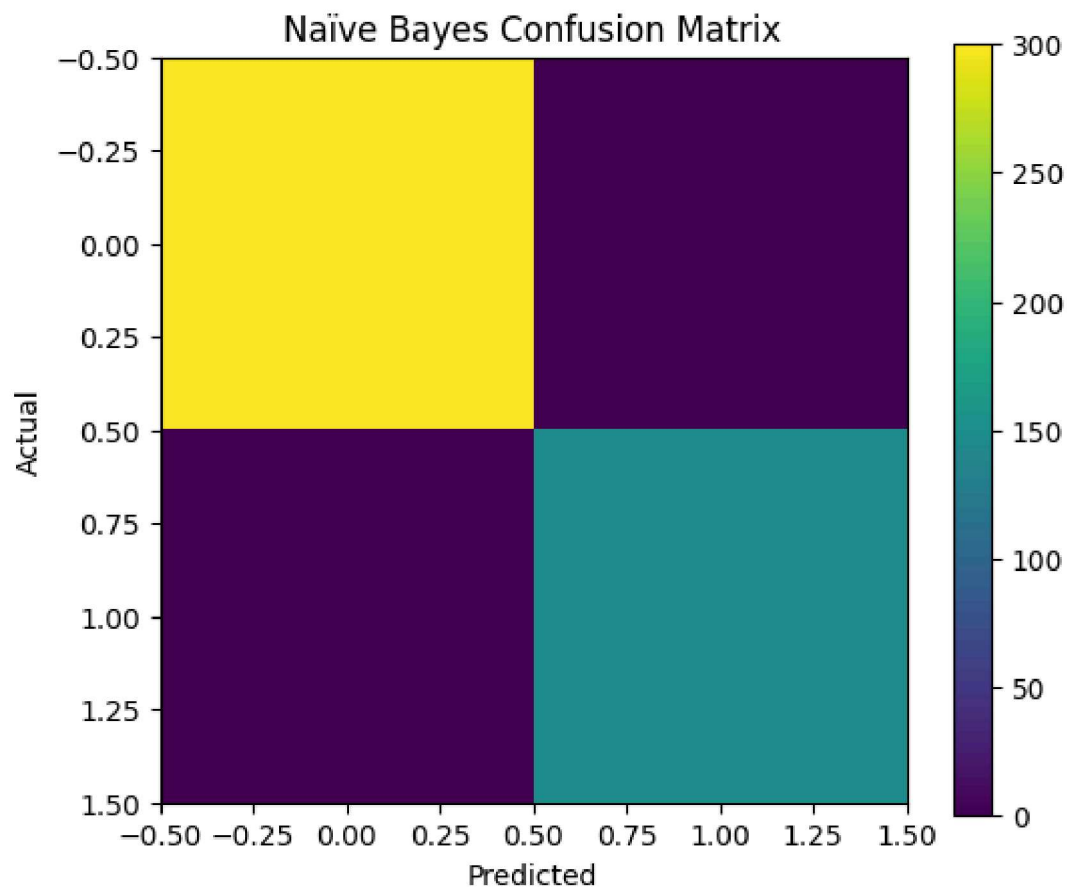
plt.imshow(cm)

plt.title("Naïve Bayes Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.colorbar()

plt.show()

print(cm)
```



```
[[300  0]
 [  0 150]]
```

```
features = network_data[
    ['PacketSize',
     'Duration',
     'Connections',
     'FailedLogins']
]

em_model = GaussianMixture(
    n_components=2,
    random_state=42
)

em_model.fit(features)

clusters = em_model.predict(features)

network_data['Cluster'] = clusters

network_data.head()
```

	PacketSize	Duration	Connections	FailedLogins	Attack	Cluster	
0	539.737132	26.996777	7.974465	-0.907808	0	0	
1	488.938856	24.623168	9.566444	0.139615	0	0	
2	551.815083	20.298152	7.622740	0.586394	0	0	
3	621.842389	16.765316	9.076115	2.887688	0	0	
4	481.267730	23.491117	4.319156	1.556553	0	0	

Next steps:

[Generate code with network_data](#)

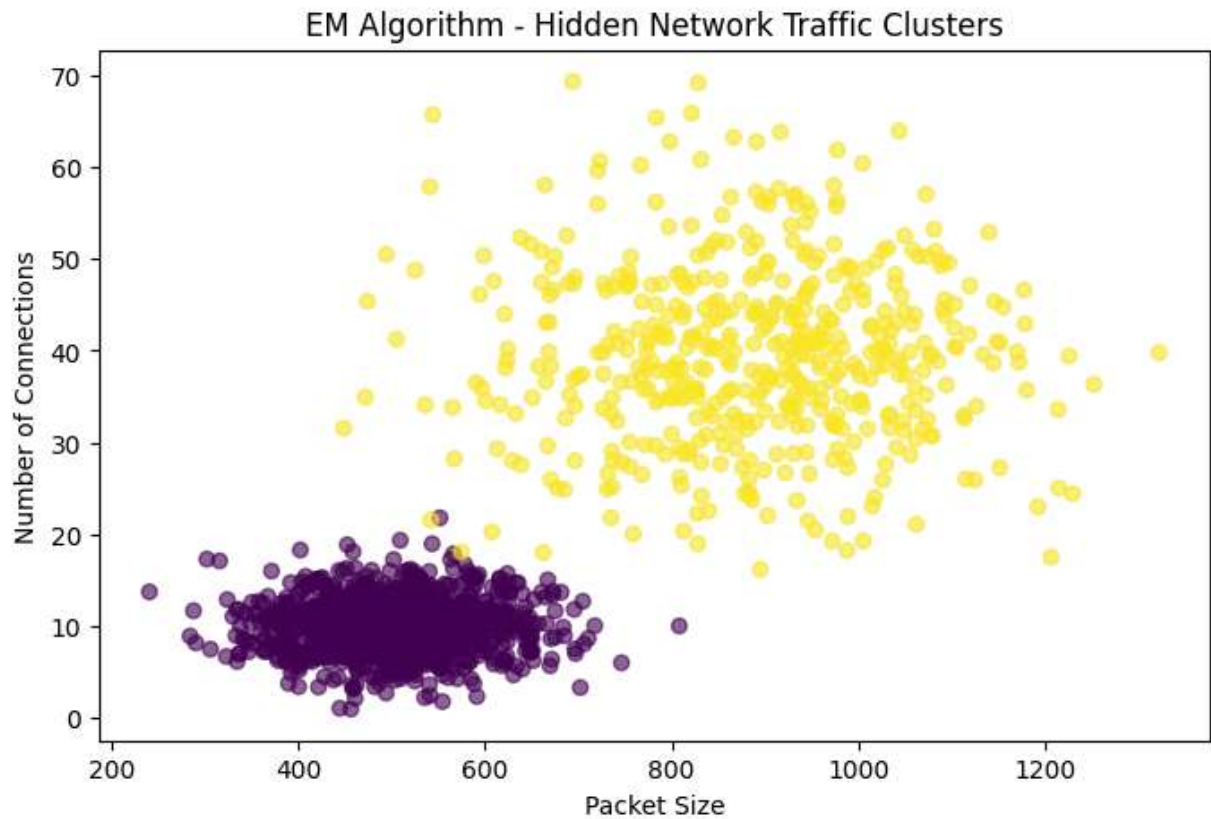
[New interactive sheet](#)

```
plt.figure(figsize=(8, 5))

plt.scatter(
    network_data['PacketSize'],
    network_data['Connections'],
    c=network_data['Cluster'],
    alpha=0.6
)

plt.xlabel("Packet Size")
plt.ylabel("Number of Connections")
plt.title("EM Algorithm - Hidden Network Traffic Clusters")

plt.show()
```



3. Agriculture – Bayesian Crop Yield Prediction 3.1 Objective

The objective is to predict crop yield under uncertain environmental conditions using:

Rainfall Temperature Soil quality Humidity Irrigation Fertilizer usage Crop variety
Historical yield

The output is:

Predicted Crop Yield

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import BayesianRidge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
np.random.seed(42)

n = 1000

rainfall = np.random.normal(700, 150, n)
temperature = np.random.normal(27, 3, n)
```

```

soil_quality = np.random.uniform(40, 100, n)
fertilizer = np.random.uniform(20, 100, n)
irrigation = np.random.uniform(0, 100, n)

yield_value = (
    0.02 * rainfall
    - 0.5 * (temperature - 27)**2
    + 0.05 * soil_quality
    + 0.03 * fertilizer
    + 0.02 * irrigation
    + np.random.normal(0, 2, n)
)

agri_data = pd.DataFrame({
    'Rainfall': rainfall,
    'Temperature': temperature,
    'SoilQuality': soil_quality,
    'Fertilizer': fertilizer,
    'Irrigation': irrigation,
    'Yield': yield_value
})

agri_data.head()

```

	Rainfall	Temperature	SoilQuality	Fertilizer	Irrigation	Yield
0	774.507123	31.198066	64.426389	44.193561	66.887653	11.348197
1	679.260355	29.773901	43.960591	65.072669	79.865578	14.676741
2	797.153281	27.178891	60.929232	84.304391	93.275294	21.995606
3	928.454478	25.059190	46.659886	30.971879	2.013785	17.122801
4	664.876994	29.094670	88.494113	66.455927	15.378371	18.121343



Next steps: [Generate code with agri_data](#) [New interactive sheet](#)

```

X = agri_data[
    ['Rainfall',
     'Temperature',
     'SoilQuality',
     'Fertilizer',
     'Irrigation']
]

y = agri_data['Yield']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,

```

```
random_state=42  
,
```

```
bayesian_model = BayesianRidge()  
  
bayesian_model.fit(X_train, y_train)  
  
y_pred = bayesian_model.predict(X_test)
```

```
mae = mean_absolute_error(y_test, y_pred)  
mse = mean_squared_error(y_test, y_pred)  
rmse = np.sqrt(mse)  
r2 = r2_score(y_test, y_pred)  
  
print("Bayesian Crop Yield Prediction")  
print("-----")  
print("MAE :", mae)  
print("MSE :", mse)  
print("RMSE :", rmse)  
print("R² :", r2)
```

```
Bayesian Crop Yield Prediction  
-----  
MAE : 4.8244708160015355  
MSE : 47.75058344817431  
RMSE : 6.910179697241911  
R² : 0.1723872306437042
```

```
plt.figure(figsize=(8, 5))  
  
plt.scatter(y_test, y_pred)  
  
plt.xlabel("Actual Yield")  
plt.ylabel("Predicted Yield")  
plt.title("Actual vs Predicted Crop Yield")  
  
plt.grid(True)  
  
plt.show()
```